

Nama : Rani Dwi Setiani
NIM : 224308043
Kelas : 6B
Mata Kuliah : Praktikum Kontrol Cerdas
Materi : Kontrol PID
Prodi : Perkeretaapian

Program transfer fuction :

```
clear
clc
% Parameter Motor
R = 0.4;
L = 2.7;
J = 0.0004;
B = 0.0022;
Ke = 0.015;
Kt = 0.05;

% Numerator dan Denominator
num = Ke;
den = [L * J (L * B + R * J) (R * B + Ke * Kt)];

% Transfer Function (Open-loop)
Gs = tf(num, den);

% PID parameters
kp = 53.7208;
Ti = 0.73245;
Td = 0.17711;
ki = kp / Ti;
kd = kp * Td;

% Buat controller PID
PID = pid(kp, ki, kd);

% Closed-loop transfer function (Negative feedback, unity)
T = feedback(PID * Gs, 1);

% Step response
step(T)
```

```

title('Step Response with PID Controller')
grid on

% Hitung karakteristik step response
info = stepinfo(T)

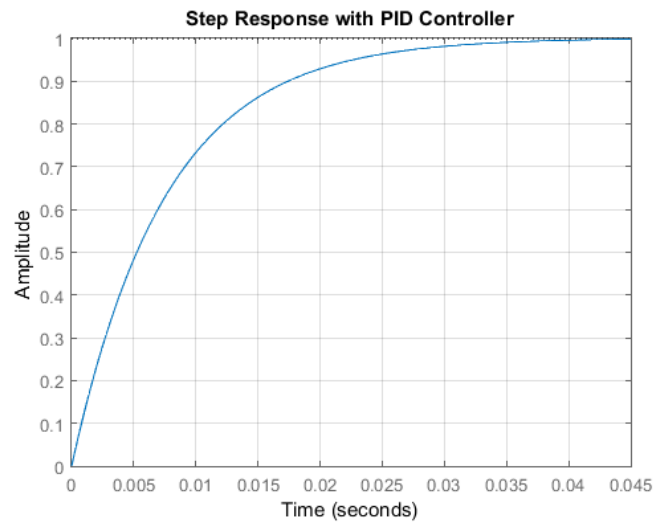
% Tampilkan hasil karakteristik
fprintf('\nKarakteristik Step Response:\n');
fprintf('Rise Time    : %.4f s\n', info.RiseTime);
fprintf('Peak Time     : %.4f s\n', info.PeakTime);
fprintf('Overshoot      : %.2f %%\n', info.Overshoot);
fprintf('Settling Time : %.4f s\n', info.SettlingTime);

```

Analisis Program transfer fuction

Program ini menggunakan kontroler PID dengan menghitung besar tegangan yang harus diberikan ke motor, berdasarkan selisih antara kecepatan target dan kecepatan actual, dan mengontrol kecepatan motor DC agar bisa mencapai kecepatan target dengan cepat, tanpa naik turun (overshoot) berlebihan, dan tetap stabil. Coding dibuat di MATLAB dengan menginputkan parameter seperti hambatan (R), Induktansi (L), Inersia (J), Gesekan (B), Konstanta tegangan balik (Ke), Konstanta toris (Kt). Selanjutnya dibuat fungsi transfer dengan model matematis dari motor. Selanjutnya masukkan nilai PID. PID merupakan singkatan dari P (Proportional) adalah seberapa besar reaksi terhadap kesalahan sekarang. I (Integral) adalah akumulasi kesalahan dari waktu ke waktu. D (Derivative) adalah prediksi kesalahan ke depan. Input dari nilai k_p , k_i , k_d adalah angkanya. Dan dibuat kontroler otomatis dari angka PID tadi dengan menginputkan **PID = pid(k_p , k_i , k_d);**. Selanjutnya gabungkan semua antara controller dan motor. Dan dibuat perintah *response* dengan menginputkan **title('Step Response with PID Controller')**. Selanjutnya program akan menghitung konerja system dengan menginputkan **info = stepinfo(T)** yang menghasilkan nilai Rise Time (TR), Overshoot, Peak Time (TP), Settling Time (TS). Rise time adalah waktu dari 0% ke 90%, overshoot adalah seberapa “kelewatan”, peak time adalah waktu tertinggi, dan settling time adalah kapan mulai stabil. Selanjutnya di cetak hasilnya agar mudah dibaca.

Hasil Run :



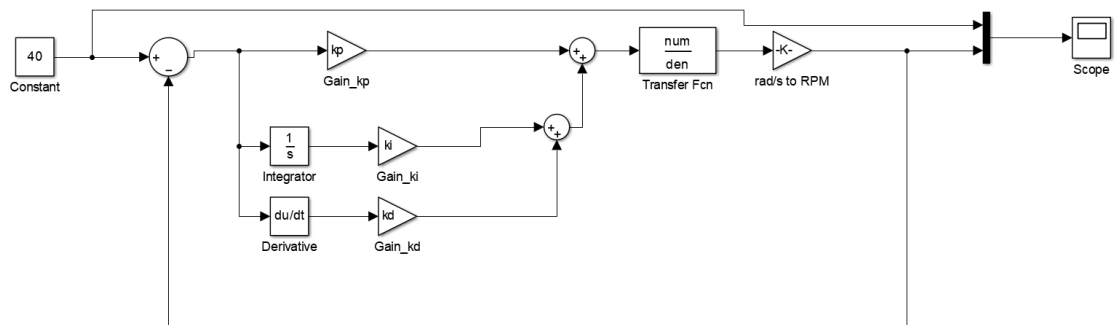
info :	Karakteristik Step Response:
RiseTime: 0.0166	Rise Time : 0.0166 s
SettlingTime: 0.0294	Peak Time : 0.0554 s
SettlingMin: 0.9002	Overshoot : 0.10 %
SettlingMax: 1.0010	Settling Time : 0.0294 s
Overshoot: 0.1002	
Undershoot: 0	
Peak: 1.0010	
PeakTime: 0.0554	

Analisis Hasil Running

Hasil *running* menghasilkan nilai Rise Time (TR), Overshoot, Peak Time (TP), Settling Time (TS). Rise time adalah waktu dari 0% ke 90%, overshoot adalah seberapa “kelewatan”, peak time adalah waktu tertinggi, dan settling time adalah kapan mulai stabil. Selanjutnya di cetak hasilnya agar mudah dibaca. *Output* dari program tersebut menghasilkan grafik *step respons* dan menghasilkan karakteristik kinerja system, yang menghasilkan karakteristik *step respons*, *Rise Time* : 0.0166 s, *Peak Time* : 0.0554 s, *Overshoot* : 0.10 %, *Settling Time* : 0.0294 s. Dari hasil tersebut dapat disimpulkan nilai *Rise Time* sangat cepat merespons saat diberi perintah, *Peak Time* adalah menghasilkan nilai tertinggi dengan cepat, tapi karena *overshoot*nya sangat kecil *peak timenya* hamper sama dengan nilai aslinya, nilai

Overshoot menghasilkan output sangat kecil sebesar 0.1% yang menunjukkan system nyaris tidak melewati nilai target, tetapi overshoot yang kecil sangat bagus karena tidak membahayakan sistem, tidak membuat gerakan motor jadi agresif dan menandakan control PID sangat stabil, selanjutnya terdapat nilai *Settling Time* menghasilkan nilai sangat cepat sebesar 0.029 s sangat stabil, tidak naik turun lagi yang menunjukkan bahwa sistem cepat menyesuaikan diri, tidak berosilasi atau lambat menstabilkan. Hasil dari program tersebut menunjukkan system naik sangat cepat, nyaris tidak *overshoot*, cepat stabil dan control PID sudah optimal.

Simulink :

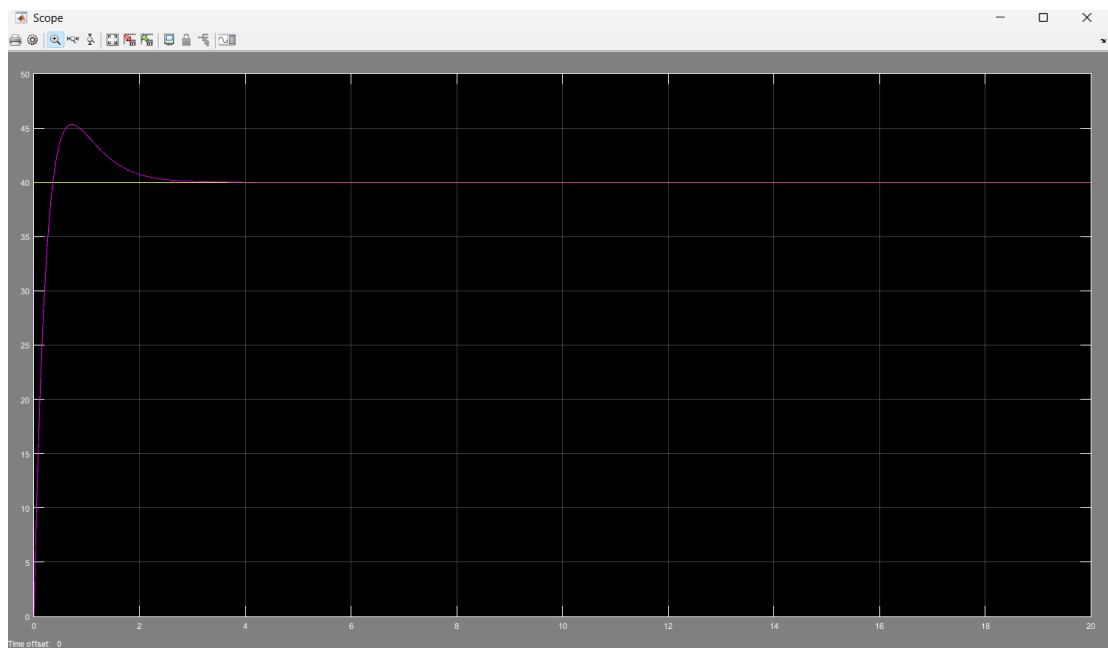


Analisis Simulink

Rangkaian ini mensimulasikan sistem kendali kecepatan motor DC menggunakan kontrol PID. Input-nya adalah kecepatan target (dalam satuan RPM), dan sistem mencoba mengatur kecepatan motor agar mengikuti target tersebut dengan baik. Terdapat **Blok Constant (40)** nilai 40 adalah *setpoint* atau perintah kecepatan. Selanjutnya terdapat Blok substraktor yang berfungsi untuk menghitung eror dari target – actual, eror ini adalah input utama ke sistem PID. **PID** berfungsi untuk mengatur sinyal kendali agar eror diperkecil. Terdapat tiga jalur PID dari cabang paralel. Cabang pertama dari komponen P terdapat blok Simulink Gain_kp yang berfungsi respons langsung terhadap eror. Cabang kedua dari komponen I terdapat blok Simulink 1/s (Integrator) Gain_kp yang berfungsi menangani eror jangka panjang. Cabang ketiga dari komponen D terdapat blok Simulink derivative Gain_kp yang berfungsi memprediksi arah perubahan eror. Hasil ketiganya dijumlahkan, lalu masuk ke *plant* (sistem motor). **Plant (motor)** berfungsi untuk

sistem nyata yang dikendalikan. Selanjutnya terdapat transfer function (Motor DC). Blok Transfer Fcn memiliki num/den, yaitu fungsi transfer motor yang kamu buat dari parameter R, L, J, B, Ke, dan Kt. Ini memodelkan bagaimana motor merespons tegangan input untuk menghasilkan kecepatan output. Selanjutnya terdapat loop feedback, Output kecepatan (rad/s atau RPM) dikembalikan ke pengurang (subtractor) untuk menghitung error baru secara terus-menerus. Inilah yang disebut feedback loop — prinsip dasar kontrol PID. **Feedback** berfungsi mengukur output dan mengontrol agar tetap sesuai target. Selanjutnya terdapat Scope, scope adalah layer osiloskop digital, **Scope** berfungsi untuk melihat hasil visual, dapat melihat respons kecepatan motor yang berubah terhadap waktu. Selanjutnya terdapat Gain, yang mengkonversi unit dari rad/s ke RPM.

Hasil Simulink :



Analisis Hasil Simulink

Hasil simulasi menunjukkan grafik respons sistem terhadap masukan berupa sinyal tangga (step input) sebesar 40. Grafik tersebut menggambarkan bagaimana sistem motor DC merespons perintah untuk mencapai kecepatan target dengan menggunakan kontrol PID. Dari bentuk kurva yang ditampilkan, terlihat bahwa sistem memiliki kemampuan untuk merespons dengan cepat, meskipun terdapat sedikit lonjakan (overshoot) pada saat awal pergerakan. Pada awal simulasi (sekitar 0–1 detik), kurva mengalami kenaikan yang tajam dari posisi awal nol

hingga mencapai nilai puncak sekitar 46. Hal ini menandakan bahwa sistem memiliki respons awal yang cepat, namun terdapat overshoot sekitar 6 satuan (sekitar 15% di atas nilai target). Setelah mencapai puncaknya, kurva mulai menurun dan secara bertahap mendekati nilai target, yaitu 40. Kurva mulai menjadi stabil dan tidak mengalami osilasi signifikan setelah waktu sekitar 5 detik. Stabilitas ini menunjukkan bahwa pengendali PID bekerja dengan baik dalam mengatur sistem agar kembali ke target meskipun terdapat overshoot di awal. Setelah sistem stabil, nilai output bertahan di sekitar angka 40, menunjukkan bahwa error steady state telah berhasil ditekan oleh pengendali PID. Secara umum, hasil simulasi memperlihatkan bahwa sistem dengan kontrol PID, hasilnya adalah responsif terhadap perubahan input, cepat mencapai target, namun mengalami sedikit overshoot, mampu stabil dalam waktu yang relatif singkat (settling time sekitar 5 detik), tidak mengalami getaran atau osilasi berkelanjutan.

Program ARDUINO IDE :

```
#include <WiFi.h>
#include <PubSubClient.h>

// =====
// KONFIGURASI WIFI & MQTT
// =====

const char* ssid = "TKA 6Barokah";
const char* password = "65432100";
const char* mqtt_server = "broker.hivemq.com";
const int mqtt_port = 1883;
WiFiClient espClient;
PubSubClient client(espClient);

// =====
// PIN KONFIGURASI
// =====

const int pinMotorPWM = 5;
const int pinREn = 21;
const int pinLEn = 22;
const int encoderPinA = 34;
const int encoderPinB = 35;

// =====
// VARIABEL PID & MOTOR
// =====

kp = 53.7208;
Ti = 0.73245;
Td = 0.17711;
```

```

ki = kp / Ti;
kd = kp * Td;
float error = 0, lastError = 0, integral = 0;
float targetSpeedKmh = 0;
float actualSpeedKmh = 0;
int pidPWM = 0;
int throttleLevel = 0;
String motorDirection = "forward"; // Default arah
// =====
// ENCODER
// =====
volatile long encoderCount = 0;
unsigned long lastSpeedCheck = 0;
void IRAM_ATTR encoderISR() {
    encoderCount++;
}
// =====
// WIFI DAN MQTT SETUP
// =====
void setup_wifi() {
    Serial.print("Connecting to WiFi ");
    Serial.print(ssid);
    WiFi.mode(WIFI_STA);
    WiFi.begin(ssid, password);
    while (WiFi.status() != WL_CONNECTED) {
        delay(500);
        Serial.print(".");
    }
    Serial.println();
    Serial.println("WiFi connected, IP: " + WiFi.localIP().toString());
}

void reconnect() {
    while (!client.connected()) {
        Serial.print("Connecting to MQTT...");
        String clientId = "ESP32Client-";
        clientId += String(random(0xffff), HEX);
        if (client.connect(clientId.c_str())) { // Connect tanpa user/password
            Serial.println("connected!");
            client.subscribe("motor/throttle");
            client.subscribe("motor/direction");
        } else {
            Serial.print("failed, rc=");
            Serial.print(client.state());
            Serial.println(" try again in 5 seconds");
            delay(5000);
        }
    }
}

```

```

// =====
// CALLBACK MQTT
// =====

void callback(char* topic, byte* payload, unsigned int length) {
    String msg;
    for (unsigned int i = 0; i < length; i++) {
        msg += (char)payload[i];
    }

    Serial.print("[MQTT] Topic: ");
    Serial.print(topic);
    Serial.print(" | Message: ");
    Serial.println(msg);

    if (String(topic) == "motor/throttle") {
        int throttle = msg.toInt();
        throttleLevel = constrain(throttle, 0, 3);
        switch (throttleLevel) {
            case 1: targetSpeedKmh = 5; break;
            case 2: targetSpeedKmh = 10; break;
            case 3: targetSpeedKmh = 15; break;
        }
        Serial.println("[MQTT] Throttle level set: " + String(throttleLevel));
    }

    if (String(topic) == "motor/direction") {
        msg.toLowerCase();
        if (msg == "forward" || msg == "reverse" || msg == "neutral") {
            motorDirection = msg;
            Serial.println("[MQTT] Direction set: " + motorDirection);
        }
    }
}

// =====
// SETUP
// =====

void setup() {
    Serial.begin(115200);

    pinMode(pinREn, OUTPUT);
    pinMode(pinLEn, OUTPUT);

    ledcSetup(0, 5000, 8); // 5kHz, 8-bit PWM
    ledcAttachPin(pinMotorPWM, 0);

    pinMode(encoderPinA, INPUT_PULLUP);
    pinMode(encoderPinB, INPUT_PULLUP);
}

```

```

attachInterrupt(digitalPinToInterrupt(encoderPinA), encoderISR, RISING);

setup_wifi();
client.setServer(mqtt_server, mqtt_port);
client.setCallback(callback);
}

// =====
// MENGHITUNG KECEPATAN
// =====

void updateSpeed() {
  noInterrupts();
  long count = encoderCount;
  encoderCount = 0;
  interrupts();

  float rpm = (count / 20.0) * 60.0; // 20 pulses per rotation
  float wheelCircumference = 3.14 * 0.1; // Diameter roda 10 cm
  actualSpeedKmh = (rpm * wheelCircumference * 60.0) / 1000.0;

  Serial.println("[ESP32] Actual Speed: " + String(actualSpeedKmh, 2) + " km/h");
  client.publish("motor/actual_speed", String(actualSpeedKmh, 2).c_str());
}

// =====
// KONTROL ARAH
// =====

void controlDirection() {
  if (motorDirection == "forward") {
    digitalWrite(pinREn, HIGH);
    digitalWrite(pinLEn, LOW);
  } else if (motorDirection == "reverse") {
    digitalWrite(pinREn, LOW);
    digitalWrite(pinLEn, HIGH);
  } else { // neutral
    digitalWrite(pinREn, LOW);
    digitalWrite(pinLEn, LOW);
  }
}

// =====
// PID CONTROL
// =====

void PIDControl() {
  if (motorDirection == "neutral" || targetSpeedKmh == 0) {
    ledcWrite(0, 0);
    pidPWM = 0;
    integral = 0;
    lastError = 0;
  }
}

```

```

    return;
}

error = targetSpeedKmh - actualSpeedKmh;
integral += error;
float derivative = error - lastError;
lastError = error;

float output = Kp * error + Ki * integral + Kd * derivative;
pidPWM = constrain((int)output, 0, 255);
ledcWrite(0, pidPWM);

Serial.println("[ESP32] PID PWM: " + String(pidPWM));
client.publish("motor/pid_output", String(pidPWM).c_str());
}

// =====
// LOOP UTAMA
// =====
void loop() {
    if (!client.connected()) {
        reconnect();
    }
    client.loop();

    controlDirection();

    unsigned long now = millis();
    if (now - lastSpeedCheck >= 1000) {
        lastSpeedCheck = now;
        updateSpeed();
        PIDControl();
    }
}

```

Analisis Program ARDUINO IDE

Program yang digunakan dalam program ARDUINO IDE merupakan implementasi dari sistem kontrol kecepatan motor DC berbasis mikrokontroler ESP32, yang dikombinasikan dengan komunikasi Internet of Things (IoT) menggunakan protokol MQTT. Sistem ini dirancang agar mampu mengatur kecepatan dan arah motor secara real-time berdasarkan perintah yang dikirimkan melalui jaringan WiFi. Perintah yang dikirim melalui MQTT berupa nilai throttle (tingkat kecepatan) dan arah gerak motor (maju, mundur, atau netral), yang masing-masing diterima melalui topik bernama motor/throttle dan motor/direction. Dalam implementasinya, sistem membaca nilai kecepatan aktual motor menggunakan

sensor rotary encoder. Sensor ini menghasilkan pulsa yang dihitung untuk mengetahui jumlah putaran roda dalam satuan waktu tertentu. Pulsa yang terdeteksi kemudian dikonversi menjadi nilai kecepatan dalam satuan km/jam. Nilai kecepatan aktual ini dibandingkan dengan nilai kecepatan target yang telah ditentukan berdasarkan throttle level yang diterima. Selisih antara target dan aktual inilah yang disebut sebagai error, yang kemudian digunakan sebagai input dalam pengendalian PID. Kontrol PID yang digunakan dalam program ini terdiri atas tiga komponen, yaitu proporsional (P), integral (I), dan derivatif (D). Komponen proporsional bertugas mengoreksi error saat ini, integral bertugas mengoreksi akumulasi error yang terjadi dari waktu ke waktu, dan derivatif berfungsi untuk memprediksi perubahan error ke depan. Nilai masing-masing parameter telah ditentukan sebelumnya, yaitu $K_p = 53.7208$, $K_i = 73.34$, dan $K_d = 9.51$. Hasil dari perhitungan PID kemudian digunakan untuk menentukan nilai sinyal PWM (Pulse Width Modulation) yang dikirim ke motor untuk mengatur kecepatannya. Nilai PWM ini dibatasi dalam rentang 0–255, sesuai dengan resolusi 8-bit PWM yang digunakan. Selain menerima perintah melalui MQTT, sistem ini juga melakukan pengiriman data secara berkala ke broker MQTT. Data yang dikirim meliputi nilai kecepatan aktual dan output PID dalam bentuk PWM, yang masing-masing dikirim melalui topik motor/actual_speed dan motor/pid_output. Hal ini memungkinkan pengguna untuk memantau performa sistem secara langsung menggunakan platform MQTT client seperti Node-RED, Blynk, atau aplikasi berbasis web. Secara keseluruhan, hasil dari implementasi program menunjukkan bahwa sistem mampu merespons perintah dengan baik. Motor dapat berputar sesuai arah yang ditentukan dan kecepatannya mendekati nilai target. Proses kontrol berlangsung secara tertutup (closed-loop), yang ditunjukkan oleh kemampuan sistem dalam menyesuaikan kecepatan berdasarkan umpan balik dari sensor. Sistem ini juga cukup fleksibel dan mendukung kendali jarak jauh, sehingga sangat potensial untuk diterapkan dalam sistem transportasi otomatis berskala kecil seperti kereta mini, kendaraan listrik edukatif, ataupun sistem robot bergerak.